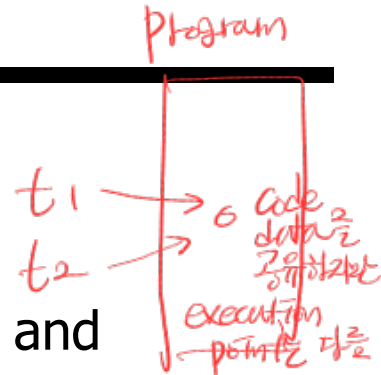
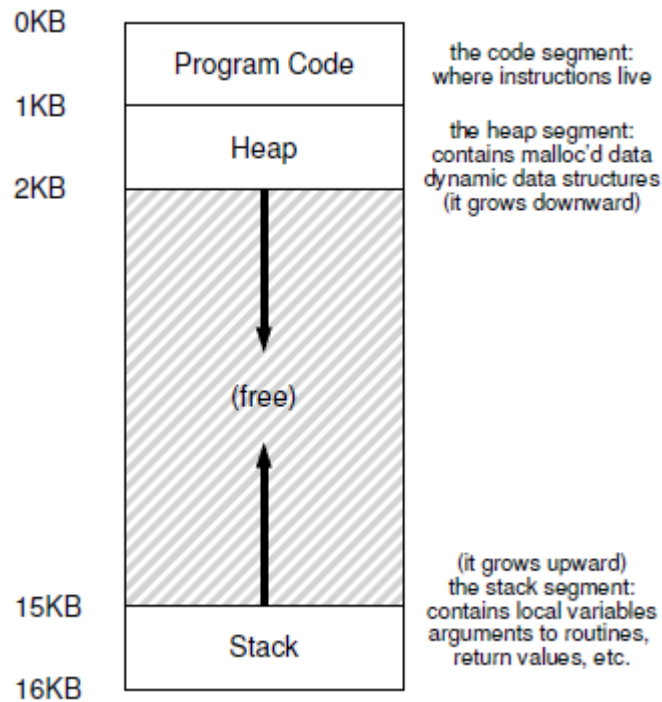

Concurrency, Thread

Thread

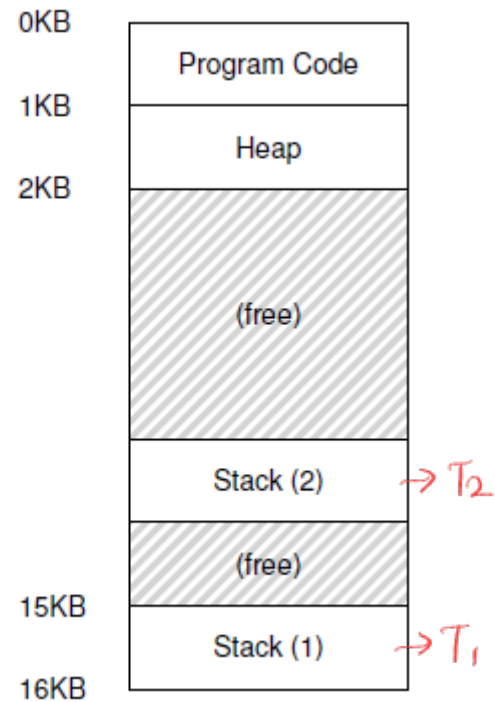
- Classic view
 - a single point of execution within a program
 - a single PC where instructions are being fetched from and executed),
- Multi-threaded program
 - Has more than one point of execution
 - multiple PCs, each of which is being fetched and executed from.
- Threads **share** the same address space and thus can access the same data *same program code*
- Each thread has its own private set of registers (including PC)
- When switching from running one (T1) to running the other (T2), a **context switch** must take place
 - thread control blocks (TCBs)
 - the address space remains the same (i.e., no need to switch the page table).



Multiple Stacks

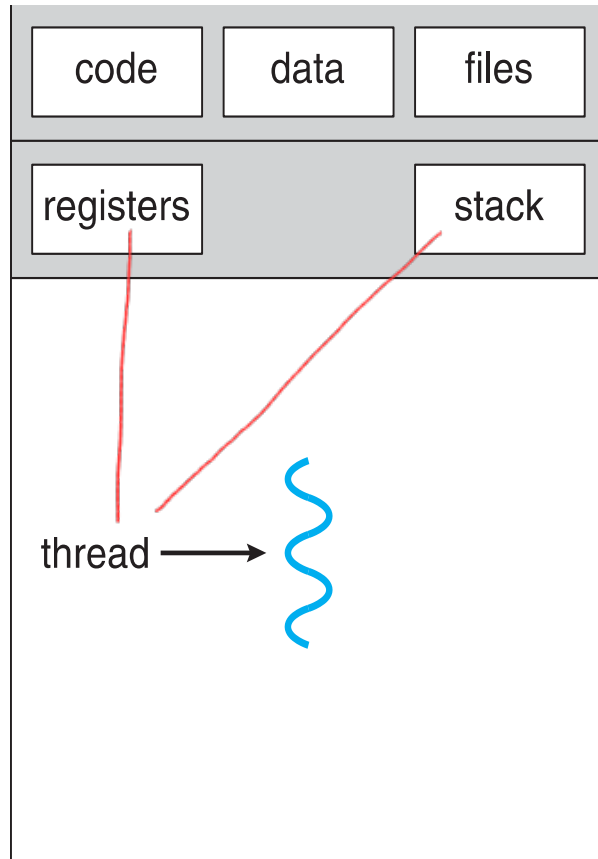


Single-Threaded

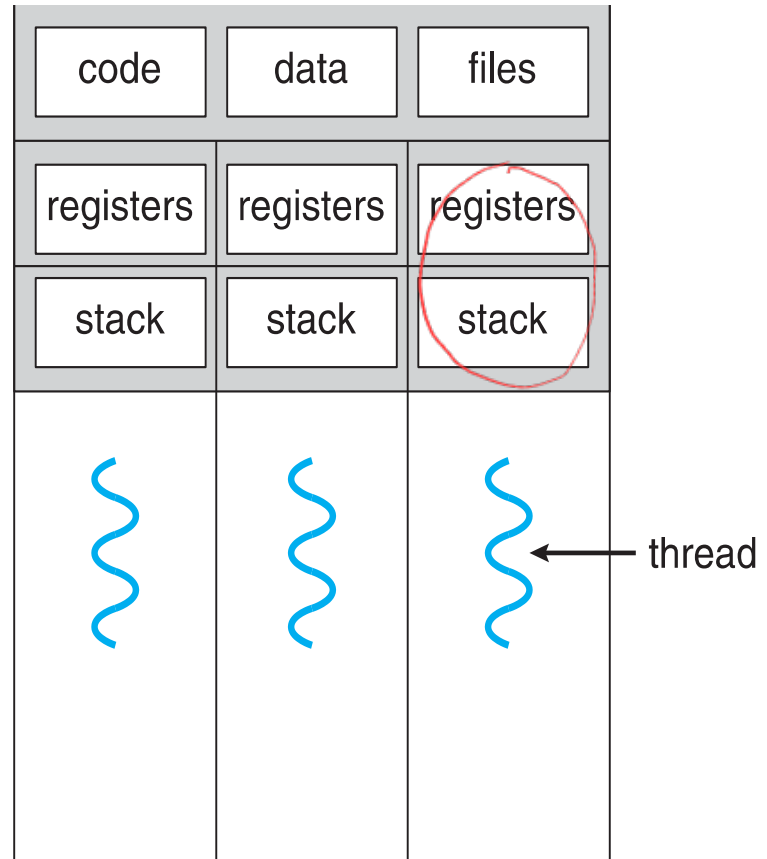


Multi-Threaded

Single and Multithreaded Processes

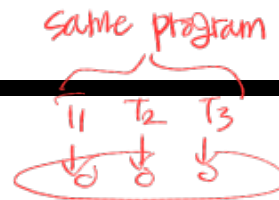


single-threaded process



multithreaded process

Why Use Threads?



- **Parallelism**

- One thread per CPU can make programs run faster on multiple processors

- **I/O overlapping (even in single processor)**

- Avoid blocking program progress due to slow I/O
- While one thread in your program waits (i.e., blocked waiting for I/O), the CPU scheduler can switch to other threads, which are ready to run and do something useful.

multi process과
비교할 때는
|
process 하나
thread를 쓰는
아니냐?

- Similar to the effect of multiprogramming

→ multi process에서 I/O를 기다리는 동안 다른 process 처리

- You could use multiple **processes** instead of **threads**.

- Threads share an address space and thus make it easy to share data
- Processes are a more sound choice for logically separate tasks

Thread

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
 - Update display
 - Fetch data
 - Spell checking
 - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

Benefits

- **Responsiveness**

- may allow continued execution if part of process is blocked, especially important for user interfaces

process 일부가 block, 스레드는 계속 실행

- **Resource Sharing**

- threads share resources of process, easier than shared memory or message passing

그리고 스레드가 block, 여전히 다른 스레드는 실행

- **Economy** *light-weight, context switching overhead*

- cheaper than process creation, thread switching lower overhead than context switching

- **Scalability**

- process can take advantage of multiprocessor architectures

Thread Creation

```
1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A"); assert(rc == 0);
16     rc = pthread_create(&p2, NULL, mythread, "B"); assert(rc == 0);
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
```

posix thread

standard library

scheduling

Once a thread is created, it may start running right away, or ready

success

이 끝날 때까지 block

pthread join() waits for a particular thread to complete

Indeterminate executions

*scheduling이
위해 순서가 보장되지 않음
main, P1, P2*

```

1  #include <stdio.h>
2  #include <assert.h>
3  #include <pthread.h>
4
5  void *mythread(void *arg) {
6      printf("%s\n", (char *) arg);
7      return NULL;
8  }
9
10 int
11 main(int argc, char *argv[]) {
12     pthread_t p1, p2;
13     int rc;
14     printf("main: begin\n");
15     rc = pthread_create(&p1, NULL, mythread, "A"); assert(rc == 0);
16     rc = pthread_create(&p2, NULL, mythread, "B"); assert(rc == 0);
17     // join waits for the threads to finish
18     rc = pthread_join(p1, NULL); assert(rc == 0);
19     rc = pthread_join(p2, NULL); assert(rc == 0);
20     printf("main: end\n");
21     return 0;
22 }
    
```

main	Thread 1	Thread2
starts running prints "main: begin" creates Thread 1 creates Thread 2 waits for T1		
	<i>Ready</i> runs prints "A" returns	
waits for T2		runs prints "B" returns
prints "main: end"		

main	Thread 1	Thread2
starts running prints "main: begin" creates Thread 1		
	runs prints "A" returns	
creates Thread 2		runs prints "B" returns
waits for T1 <i>returns immediately; T1 is done</i> waits for T2 <i>returns immediately; T2 is done</i> prints "main: end"		

main	Thread 1	Thread2
starts running prints "main: begin" creates Thread 1 creates Thread 2		
		runs prints "B" returns
waits for T1	runs prints "A" returns	
waits for T2 <i>returns immediately; T2 is done</i> prints "main: end"		

Why It Gets Worse: Shared Data

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include "mythreads.h"
4
5  static volatile int counter = 0;
6
7  //
8  // mythread()
9  //
10 // Simply adds 1 to counter repeatedly, in a loop
11 // No, this is not how you would add 10,000,000 to
12 // a counter, but it shows the problem nicely.
13 //
14 void *
15 mythread(void *arg)
16 {
17     printf("%s: begin\n", (char *) arg);
18     int i;
19     for (i = 0; i < 1e7; i++) {
20         counter = counter + 1;
21     }
22     printf("%s: done\n", (char *) arg);
23     return NULL;
24 }
25
```

Handwritten notes:
- Red circle around `counter`
- Red arrows pointing to `counter` from `main`, `P1`, and `P2`
- Red text "shared data" with an arrow pointing to the `counter` variable

```
prompt> gcc -o main main.c -Wall -pthread
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 20000000)
```

[expected]

```
26 //
27 // main()
28 //
29 // Just launches two threads (pthread_create)
30 // and then waits for them (pthread_join)
31 //
32 int
33 main(int argc, char *argv[])
34 {
35     pthread_t p1, p2;
36     printf("main: begin (counter = %d)\n", counter);
37     Pthread_create(&p1, NULL, mythread, "A");
38     Pthread_create(&p2, NULL, mythread, "B");
39
40     // join waits for the threads to finish
41     Pthread_join(p1, NULL);
42     Pthread_join(p2, NULL);
43     printf("main: done with both (counter = %d)\n", counter);
44     return 0;
45 }
```

```
prompt> ./main
main: begin (counter = 0)
A: begin
B: begin
A: done
B: done
main: done with both (counter = 19345221)
```

even indeterminate

[real]

Uncontrolled Scheduling

counter = counter + 1;

C statement



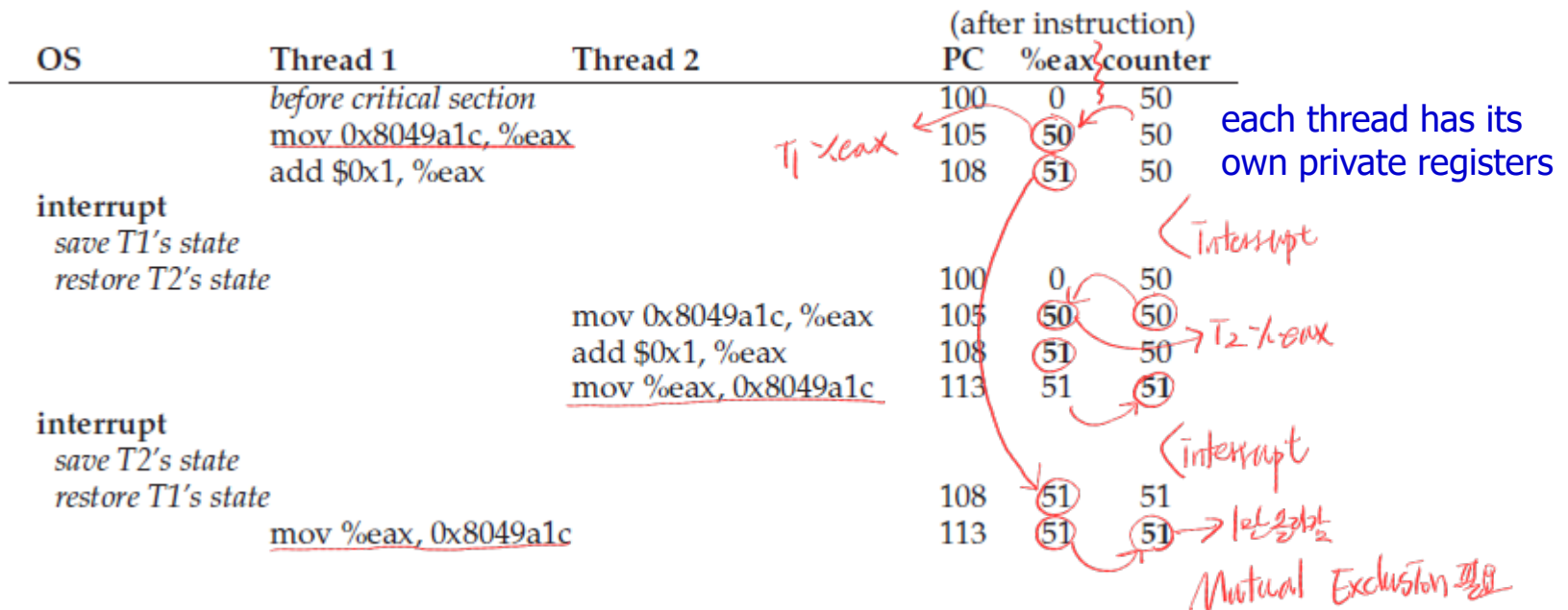
Intel x86

↓ compile

다른 thread가 끼어들 수 있음

```
100  mov 0x8049a1c, %eax
105  add $0x1, %eax
108  mov %eax, 0x8049a1c
```

3 machine instruction



Race Condition

- **Race condition**
 - Results depend on the timing execution of the code, indeterminate
- **Critical section**
 - a piece of code that accesses **a shared variable** (or more generally, **a shared resource**) and must not be concurrently executed by more than one thread.
- **Mutual exclusion.**
 - guarantees that if one thread is executing within the critical section, the others will be prevented from doing so.



Edsger Dijkstra
Turing award 1972



Wish For Atomicity

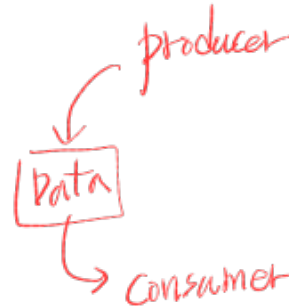
- We need more powerful instructions
 - do exactly whatever we needed done in a single step → ^{all or nothing} **atomic**
 - could not be interrupted mid-instruction, because that is precisely the guarantee we receive from the hardware
 - remove the possibility of an untimely interrupt
 - E.g. memory-add 0x8049a1c, \$0x1 (Read
Add
Write)
- Instead of many complex atomic instructions, hardware provides a few useful instructions upon which we can build a general set of **synchronization primitives**.

One More Problem: Waiting For Another

- Another common interaction
 - One thread must wait for another to complete some action before it continues
 - e.g., when a process performs a disk I/O and is put to sleep; when the I/O completes, the process needs to be roused from its slumber so it can continue.

무기대함

- **Condition variables**



Thread Libraries

- Thread library provides programmer with **API** for creating and managing threads
- Two primary ways of implementing
 - **Library** entirely in user space
 - Function call, not system call
 - **Kernel-level library** supported by the OS

system call x
user-level library clone() 등
API call로 가능

Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- *Specification*, not *implementation*
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Solaris, Linux, Mac OS X)
- Compile
 - prompt> gcc -o main main.c -Wall -pthread

Thread API

- Thread Creation
- Thread Completion
- Locks
- Condition Variables

Thread Creation

```
#include <pthread.h>
int
pthread_create(    pthread_t *    thread,
                  const pthread_attr_t * attr,
                  void *          (*start_routine) (void*),
                  void *          arg);
```

- Arguments
 - **thread**: a pointer to a structure of type `pthread_t`
 - Used to interact with this thread
 - **attr**: specify any attributes this thread might have
 - e.g., stack size or scheduling priority
 - initialized with a separate call to `pthread_attr_init()`
 - **function pointer**: which function should this thread start running in?
 - **arg**: the argument to be passed to the function where the thread begins execution
 - void pointer allows us to pass in any type of argument

Thread Creation

```
1  #include <pthread.h>
2
3  typedef struct __myarg_t {
4      int a;
5      int b;
6  } myarg_t;
7
8  void *mythread(void *arg) {
9      myarg_t *m = (myarg_t *) arg;
10     printf("%d %d\n", m->a, m->b);
11     return NULL;
12 }
13
14 int
15 main(int argc, char *argv[]) {
16     pthread_t p;
17     int rc;
18
19     myarg_t args;
20     args.a = 10;
21     args.b = 20;
22     rc = pthread_create(&p, NULL, mythread, &args);
23     ...
24 }
```

unpack the arguments as desired

package into a single type



Thread Completion

```
int pthread_join(pthread_t thread, void **value_ptr);
```

- Wait for a thread to complete
- Arguments
 - pthread_t: specify which thread to wait for.
 - a pointer to the return value

*print
10 20
returned 1 2*

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <assert.h>
4  #include <stdlib.h>
5
6  typedef struct __myarg_t {
7      int a;
8      int b;
9  } myarg_t;
10
11 typedef struct __myret_t {
12     int x;
13     int y;
14 } myret_t;
15
16 void *mythread(void *arg) {
17     myarg_t *m = (myarg_t *) arg;
18     printf("%d %d\n", m->a, m->b);
19     myret_t *r = Malloc(sizeof(myret_t));
20     r->x = 1;
21     r->y = 2;
22     return (void *) r;
23 }
24
25 int
26 main(int argc, char *argv[]) {
27     pthread_t p;
28     myret_t *m;
29
30     myarg_t args = {10, 20};
31     Pthread_create(&p, NULL, mythread, &args);
32     Pthread_join(p, (void **) &m);
33     printf("returned %d %d\n", m->x, m->y);
34     free(m);
35     return 0;
36 }
```

Thread Completion

- don't return a pointer which refers to something allocated on the thread's call stack
 - automatically deallocated *function이 끝났기 때문*

```
1 void *mythread(void *arg) {
2     myarg_t *m = (myarg_t *) arg;
3     printf("%d %d\n", m->a, m->b);
4     myret_t r; // ALLOCATED ON STACK: BAD!
5     r.x = 1; local variable in stack
6     r.y = 2;
7     return (void *) &r;
8 }
```

*malloc으로 heap에
할당해서 주어야 됨*

Locks

```
int pthread_mutex_lock(pthread_mutex_t *mutex);  
int pthread_mutex_unlock(pthread_mutex_t *mutex);
```

- Providing mutual exclusion to a critical section via **locks**

```
pthread_mutex_t lock;  
pthread_mutex_lock(&lock);  
x = x + 1; // or whatever your critical section is  
pthread_mutex_unlock(&lock);
```

lock
~~~~~) *critical*
unlock *section*

- If no other thread holds the lock, the thread will acquire the lock and enter the critical section.
- If another thread does indeed hold the lock, the thread trying to grab the lock will not return from the call/until it has acquired the lock
- Lock initialization
 - Static `pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;`
 - Dynamic `int rc = pthread_mutex_init(&lock, NULL);`
`assert(rc == 0); // always check success!`
- Lock destroy
 - `pthread_mutex_destroy()`

trylock

```
int pthread_mutex_trylock(pthread_mutex_t *mutex);  
int pthread_mutex_timedlock(pthread_mutex_t *mutex,  
                             struct timespec *abs_timeout);
```

- Returns failure if the lock is already held *o/b lock already held*
- the timedlock version of acquiring a lock returns after a timeout or after acquiring the lock, whichever happens first.
- avoid getting stuck (perhaps indefinitely) in a lock acquisition routine (prevent deadlock).

Condition Variables

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex);  
int pthread_cond_signal(pthread_cond_t *cond);
```

- To use a condition variable, one has to in addition have a lock that is associated with this condition
 - Prevent a race condition
- Wait
 - puts the calling thread to sleep, and thus waits for some other thread to signal it, usually when something in the program has changed that the now-sleeping thread might care about.
 - Release the lock before sleep, re-acquires the lock before returning
 - For safety, the waiting thread has to re-check the condition after wake-up (There could be a spurious wake-up)

7/22/21

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

```
Pthread_mutex_lock(&lock);  
while (ready == 0) {  
    pthread_cond_wait(&cond, &lock);  
} CS  
Pthread_mutex_unlock(&lock);
```

```
Pthread_mutex_lock(&lock);  
ready = 1;  
Pthread_cond_signal(&cond);  
Pthread_mutex_unlock(&lock);
```


Spin

```
while (ready == 0); // spin
```

instead of wait()

```
ready = 1;
```

instead of signal()

- A simple flag instead of a condition variable and associated lock
- Don't ever do this *condition variable 대신 플래그 flag*
 - Poor performance: spinning for a long time just wastes CPU cycles
 - Error prone
아마도 나는 잠깐만 쉬고

User Threads and Kernel Threads

- **User threads**

- management done by user-level threads library
- Three primary thread libraries:
 - POSIX **Pthreads**
 - Windows threads
 - Java threads

- **Kernel threads**

- Supported by the Kernel
- Examples – virtually all general-purpose operating systems, including: Windows, Solaris, Linux, Tru64 UNIX, Mac OS X

Implicit Threading *↔ Explicit: create destroy*

- **Growing in popularity as numbers of threads increase, program correctness more difficult with explicit threads**
- **Creation and management of threads done by compilers and run-time libraries rather than programmers**
- **Three methods explored**
 - Thread Pools
 - OpenMP
- **Other methods**
 - Intel Threading Building Blocks (TBB): Parallel C++ program
 - `java.util.concurrent` package

Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
 - Usually slightly faster to service a request with an existing thread than create a new thread *마찬가지로 thread를 사용하는게 더 빠름*
 - Allows the number of threads in the application(s) to be bound to the size of the pool *thread의 개수를 pool의 크기로 제한*
 - Separating task to be performed from mechanics of creating task allows different strategies for running task *thread를 만들어 task 할당X*
 - i.e. Tasks could be scheduled to run periodically

```
public class ThreadExample {  
    public static void ThreadProc() {  
        ...  
    }  
  
    public static void Main() {  
        Thread t = new Thread(  
            new ThreadStart(ThreadProc));  
        t.Start();  
        ...  
        t.Join();  
    }  
}
```

직접 thread 생성

```
public class Example {  
    public static void Main()  
    {  
        ThreadPool.QueueUserWorkItem(ThreadProc);  
    }  
  
    static void ThreadProc(Object stateInfo)  
    {  
        ...  
    }  
}
```

.NET example

OpenMP

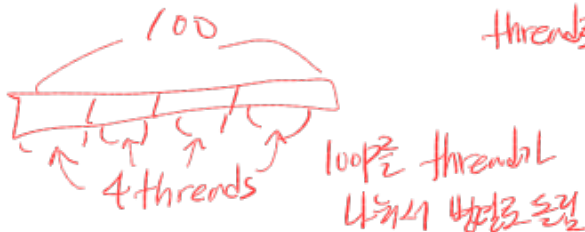
- Set of compiler directives and an API for C, C++, FORTRAN
- Provides support for parallel programming in shared-memory environments
- Identifies **parallel regions** – blocks of code that can run in parallel

`#pragma omp parallel`

Create as many threads/as there are cores

```
#pragma omp parallel for
for(i=0;i<N;i++) {
    c[i] = a[i] + b[i];
}
```

Run for loop in parallel



```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    /* sequential code */

    compiler → #pragma omp parallel
    {
        이 블록을 threads 2개씩
        { printf("I am a parallel region.");
        }

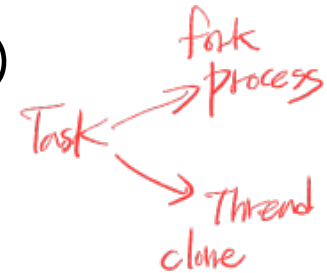
        /* sequential code */

        return 0;
    }
```

Linux Threads

- Linux refers to them as *tasks* rather than *threads*
- Thread creation is done through `clone()` system call
- `clone()` allows a child task to share the address space of the parent task (process)
 - for thread creation (cf> fork for process creation)
 - Flags control behavior

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.



fork() share 资源
→ only address space

- `struct task_struct` points to process data structures (shared or unique)

Homework

- Homework in Chap 27 (Debugging Race/Deadlock w/ *helgrind*)

Quiz

Join - main 함수가 thread가 exit할 때까지 block될
↳ stack memory leak 방지

1. A race condition can occur when multiple processes or threads execute in their critical sections simultaneously. (O)
2. Mutual exclusion ensures that only one process or thread can enter its critical section at any given time. (O)
3. When you don't care when or how the thread finishes, you can use pthread_detach(). (O) *main thread가 관제를 끝내버림 → Join할 필요 X*
4. Two threads can run the same function concurrently using pthread_create(). (O)
5. A thread created by pthread_create() starts running ~~immediately~~ after the function returns. (X) *→ ready 상태로 존재 (scheduling이 먼저 수행된 뒤 결정)*
6. Mutexes in pthreads are used to protect shared resources from concurrent access. (O)
7. pthread_mutex_trylock() ~~blocks~~ the calling thread until the mutex becomes available. (X) *상대하면 즉시 return → block되지 않음*
8. When using clone() with the CLONE_VM flag, the child gets ~~a copy~~ of the parent's memory space. (X) *share*
9. OpenMP is a shared-memory parallel programming model that uses compiler directives. (O)
10. NPTL is a threading library that implements the POSIX pthreads API. (O)
*↳ user-level library (시스템 커널)
user는 POSIX thread API를 보고 programming하면 됨*

pthread example 1 – join version

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
```

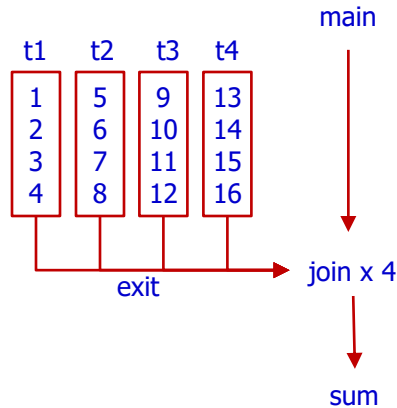
```
#define NUM_THREADS 4
#define ARRAY_SIZE 16
```

```
int array[ARRAY_SIZE];
int partial_sums[NUM_THREADS];
```

```
typedef struct {
    int thread_id;
    int start_idx;
    int end_idx;
} thread_arg_t;
```

```
void* partial_sum(void* arg) {
    thread_arg_t* data = (thread_arg_t*)arg;
    int sum = 0;
    for (int i = data->start_idx; i < data->end_idx; ++i) {
        sum += array[i];
    }
    partial_sums[data->thread_id] = sum;
    pthread_exit(NULL);
}
```

```
int main() {
    pthread_t threads[NUM_THREADS];
    thread_arg_t args[NUM_THREADS];
    int segment_size = ARRAY_SIZE / NUM_THREADS;
```



```
// Initialize array with some values
for (int i = 0; i < ARRAY_SIZE; ++i) {
    array[i] = i + 1; // array = [1, 2, ..., 16]
}
```

```
// Create threads
for (int t = 0; t < NUM_THREADS; ++t) {
    args[t].thread_id = t;
    args[t].start_idx = t * segment_size;
    args[t].end_idx = (t == NUM_THREADS - 1) ?
        ARRAY_SIZE : (t + 1) * segment_size;
    pthread_create(&threads[t], NULL, partial_sum, &args[t]);
}
```

```
// Wait for all threads to complete 모든 thread 기다림
for (int t = 0; t < NUM_THREADS; ++t) {
    pthread_join(threads[t], NULL);
}
```

```
// Combine partial sums
int total_sum = 0;
for (int t = 0; t < NUM_THREADS; ++t) {
    total_sum += partial_sums[t];
}
```

```
printf("Total sum = %d\n", total_sum); // Expected: 136
```

```
return 0;
}
```

pthread example 2 – cond version

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUM_THREADS 4
#define ARRAY_SIZE 16

int array[ARRAY_SIZE];
int partial_sums[NUM_THREADS];
int finished_threads = 0;

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;

typedef struct {
    int thread_id;
    int start_idx;
    int end_idx;
} thread_arg_t;

void* partial_sum(void* arg) {
    thread_arg_t* data = (thread_arg_t*)arg;
    int sum = 0;

    for (int i = data->start_idx; i < data->end_idx; ++i) {
        sum += array[i];
    }

    partial_sums[data->thread_id] = sum;

    pthread_mutex_lock(&lock);
    finished_threads++;
    if (finished_threads == NUM_THREADS) {
        pthread_cond_signal(&cond);
    }
    pthread_mutex_unlock(&lock);

    pthread_exit(NULL);
}
```

```
int main() {
    pthread_t threads[NUM_THREADS];
    thread_arg_t args[NUM_THREADS];
    int segment_size = ARRAY_SIZE / NUM_THREADS;

    // Initialize the array
    for (int i = 0; i < ARRAY_SIZE; ++i) array[i] = i + 1;

    // Create worker threads
    for (int t = 0; t < NUM_THREADS; ++t) {
        args[t].thread_id = t;
        args[t].start_idx = t * segment_size;
        args[t].end_idx = (t == NUM_THREADS - 1) ? ARRAY_SIZE : (t + 1) * segment_size;
        pthread_create(&threads[t], NULL, partial_sum, &args[t]);
    }

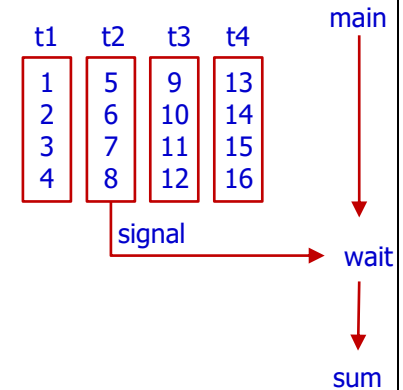
    // Wait until all threads finish using a condition variable
    pthread_mutex_lock(&lock);
    while (finished_threads < NUM_THREADS) {
        pthread_cond_wait(&cond, &lock);
    }
    pthread_mutex_unlock(&lock);

    // Sum the results
    int total_sum = 0;
    for (int t = 0; t < NUM_THREADS; ++t) total_sum += partial_sums[t];

    printf("Total sum = %d\n", total_sum); // Expected: 136

    // Cleanup
    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cond);

    return 0;
}
```



pthread example 3 – barrier version

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
```

```
#define NUM_THREADS 4
#define ARRAY_SIZE 16
```

```
int array[ARRAY_SIZE];
int partial_sums[NUM_THREADS];
int total_sum = 0;
pthread_barrier_t barrier;
```

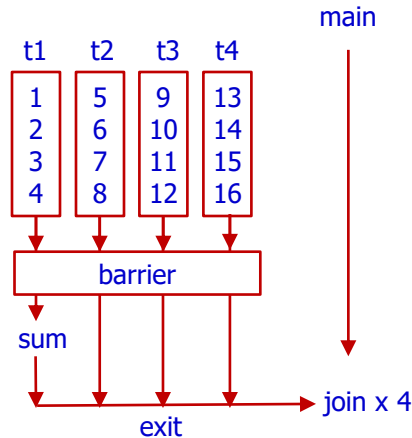
```
typedef struct {
    int thread_id;
    int start_idx;
    int end_idx;
} thread_arg_t;
```

```
void* partial_sum(void* arg) {
    thread_arg_t* data = (thread_arg_t*)arg;
    int sum = 0;

    for (int i = data->start_idx; i < data->end_idx; ++i) sum += array[i];
    partial_sums[data->thread_id] = sum;

    // Wait at the barrier until all threads finish their partial sums
    pthread_barrier_wait(&barrier);

    // Only one thread (e.g., thread 0) aggregates the result
    if (data->thread_id == 0) {
        for (int t = 0; t < NUM_THREADS; ++t) total_sum += partial_sums[t];
        printf("Total sum = %d\n", total_sum); // Expected: 136
    }
}
```



```
pthread_exit(NULL);
}
```

```
int main() {
    pthread_t threads[NUM_THREADS];
    thread_arg_t args[NUM_THREADS];
    int segment_size = ARRAY_SIZE / NUM_THREADS;

    // Initialize array
    for (int i = 0; i < ARRAY_SIZE; ++i) array[i] = i + 1;

    // Initialize the barrier for NUM_THREADS
    pthread_barrier_init(&barrier, NULL, NUM_THREADS);

    // Create threads
    for (int t = 0; t < NUM_THREADS; ++t) {
        args[t].thread_id = t;
        args[t].start_idx = t * segment_size;
        args[t].end_idx = (t == NUM_THREADS - 1) ?
            ARRAY_SIZE : (t + 1) * segment_size;
        pthread_create(&threads[t], NULL, partial_sum, &args[t]);
    }

    // Wait for threads to finish
    for (int t = 0; t < NUM_THREADS; ++t) {
        pthread_join(threads[t], NULL);
    }

    // Cleanup
    pthread_barrier_destroy(&barrier);
    return 0;
}
```

pthread example 3 – OpenMP version

```
#include <stdio.h>
#include <omp.h>
```

```
#define ARRAY_SIZE 16
```

```
int main() {
    int array[ARRAY_SIZE];
    int total_sum = 0;
```

```
    // Initialize array with values 1 to 16
    for (int i = 0; i < ARRAY_SIZE; ++i) {
        array[i] = i + 1;
    }
```

```
    // Parallel sum using OpenMP
```

```
    #pragma omp parallel for reduction(+:total_sum)
```

```
    for (int i = 0; i < ARRAY_SIZE; ++i) {
        total_sum += array[i];
    }
```

```
    printf("Total sum = %d\n", total_sum); // Expected: 136
    return 0;
```

```
}
```

Feature	Description
#pragma omp parallel for	Tells the compiler to parallelize the for loop.
reduction(+:total_sum)	Combines partial sums from each thread into total_sum safely.
Automatic Threading	OpenMP chooses the number of threads (can be set via OMP_NUM_THREADS).